Linear Regression

Complete Guide with All Techniques

From Mathematical Foundations to Advanced Regularization & Model Diagnostics

Dataset: Boston Housing | Target: Median Home Value (medv)

- Simple & Multiple LR
- Polynomial Regression
- Ridge / Lasso / Elastic Net
- Cross-Validation & Tuning
- Assumption Diagnostics
- Statistical Validation

Agenda — What We'll Cover

1.	**Data Loading & EDA**	Understand the data before modelling
2.	**Simple Linear Regression**	One feature, OLS from scratch + sklearn
3.	**Multiple Linear Regression**	All features, interpreting coefficients
4.	**Polynomial Regression**	Non-linear curves, overfitting demo
5.	**Feature Scaling**	StandardScaler vs MinMaxScaler
6.	**Regularization**	Ridge (L2), Lasso (L1), Elastic Net
7.	**Cross-Validation & Tuning**	KFold, GridSearchCV, Learning Curves
8.	**Assumption Diagnostics**	Linearity, Normality, VIF, Homoscedasticity
9.	**Statistical Validation**	statsmodels: p-values, F-test, CI, Durbin-Watson
10.	**Model Comparison**	Final side-by-side metric table & takeaways

What is Linear Regression?

The most fundamental supervised learning algorithm for predicting continuous values

Definition

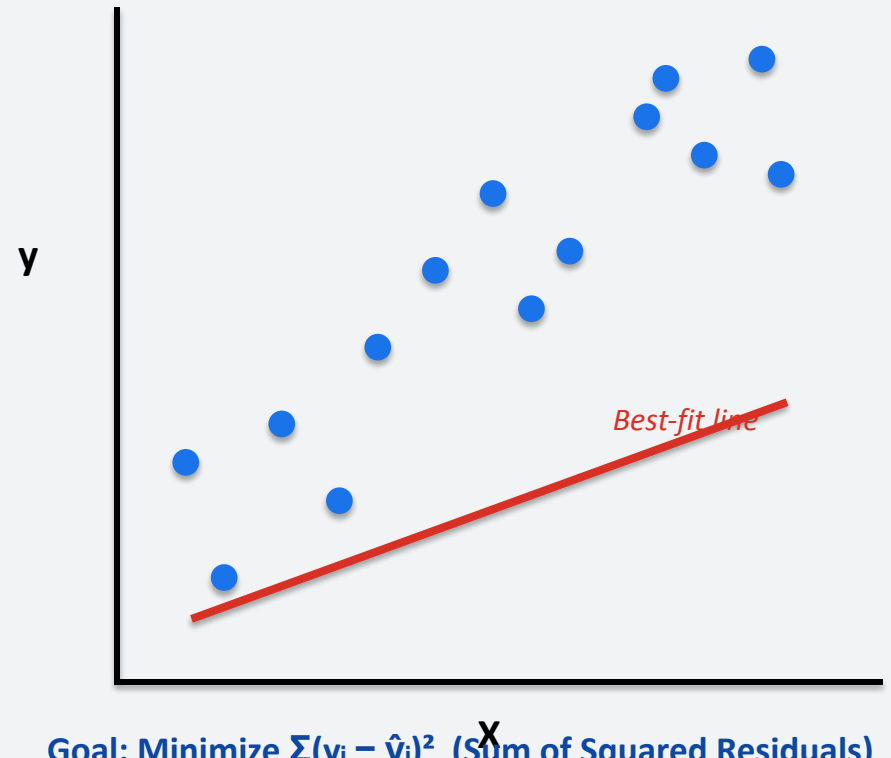
Linear Regression models the relationship between input features (X) and a continuous target variable (y) by fitting a straight line (or hyperplane) that minimizes the prediction error.

The Equation

$$\hat{y} = w_0 + w_1X_1 + w_2X_2 + \ldots + w_nX_n$$

- w_0 = intercept (bias term)
- $w_1 \ldots w_n$ = coefficients (slopes)
- $X_1 \ldots X_n$ = input features
- $\hat{y}$ = predicted value

How OLS Finds the Best Line



Goal: Minimize $\Sigma(y_i - \hat{y}_i)^2$ (Sum of Squared Residuals)

The Dataset: Boston Housing

506 observations | 13 features | Target: medv (median home value in $1000s)

Feature	Description	Type
crim	Per-capita crime rate by town	Continuous
zn	Proportion residential land >25k sqft	Continuous
indus	Non-retail business acres per town	Continuous
chas	Charles River dummy (1/0)	Binary
nox	Nitric oxide concentration	Continuous
rm	Avg rooms per dwelling	Continuous
age	% owner-occ units pre-1940	Continuous
dis	Distance to employment centres	Continuous
rad	Highway accessibility index	Discrete
tax	Property-tax rate per $10k	Continuous
ptratio	Pupil-teacher ratio	Continuous
b	Proportion measure (race)	Continuous
lstat	% lower status population	Continuous

Key Facts

- 506 rows, no missing values
- Target: medv (continuous)
- Strongest +corr: rm (+0.70)
- Strongest −corr: lstat (−0.74)
- rad & tax correlated (0.91) → multicollinearity
- medv is right-skewed, capped at $50k

Exploratory Data Analysis (EDA)

Understand distributions, correlations, and relationships before modelling

Why EDA Before Modelling?

- Spot outliers and skewed distributions
- Identify which features correlate with target
- Detect multicollinearity between features
- Determine if relationships are linear or curved

EDA Steps in the Notebook

1. Target distribution (histogram + boxplot)
2. Feature distributions (all 13 histograms)
3. Correlation heatmap (feature ↔ feature)
4. Scatter plots (top features vs target)

Key EDA Findings

- rm (rooms) has the strongest positive correlation with medv (+0.70) — more rooms → higher price
- lstat (lower status %) has the strongest negative correlation (−0.74) — higher lstat → lower price
- lstat vs medv shows a CURVED relationship → polynomial regression may help
- rad and tax are correlated with each other (r = 0.91) → watch for multicollinearity
- medv distribution is right-skewed with a cap at $50k

Data Preprocessing: Train-Test Split

Split data BEFORE modelling to evaluate generalization on unseen data

Important Points

- random_state=42 ensures reproducibility — everyone gets the same split
- Never use test data for training or feature scaling (data leakage!)
- We use a helper function evaluate_model() to consistently compute R^2, RMSE, MAE for every model
- All results are stored for the final comparison table at the end

Simple Linear Regression (Univariate)

One feature → one target, fit a straight line using Ordinary Least Squares

The Equation

$$\hat{y} = w_0 + w_1 \cdot X$$

w_0 (intercept) = value of y when X = 0

w_1 (slope) = change in y per 1-unit increase in X

OLS Formulas

Slope: $w_1 = \Sigma(X_i - \bar{X})(y_i - \bar{y}) \,/\, \Sigma(X_i - \bar{X})^2$

Intercept: $w_0 = \bar{y} - w_1 \cdot \bar{X}$

Minimizes: $\Sigma(y_i - \hat{y}_i)^2$ (Sum of Squared Residuals)

Example: Using lstat (strongest −corr with medv)

Model: medv = 34.84 + (−0.97) × lstat

Interpretation: For every 1% increase in lower-status population, home value drops by ~$970

Train R^2 = 0.542 | Test R^2 = 0.543

→ Only ~54% of variance explained — the curved residual pattern tells us a straight line isn't enough!

Multiple Linear Regression

Use ALL features to predict the target — each gets its own coefficient

The Equation

$$\hat{y} = w_0 + w_1X_1 + w_2X_2 + \ldots + w_{13}X_{13}$$

Each w_i: "Holding all else constant, how much does medv change per 1-unit increase in X_i?"

Results

Train R^2 = 0.7509

Test R^2 = 0.6688

Big improvement over Simple LR (0.54 ⟶ 0.67)!

Feature	Coefficient	Effect on Price
nox	−17.77	Strong negative
rm	+3.81	Strong positive
dis	−1.48	Negative
ptratio	−0.95	Negative
lstat	−0.52	Negative
chas	+2.69	Positive
rad	+0.31	Positive

Key Observations

- nox (pollution) has the largest negative impact
- rm (rooms) has the largest positive impact
- chas (river) adds ~$2,690 to home value
- *Coefficient magnitudes depend on feature scale — compare after scaling for fair ranking*

Polynomial Regression

Capture non-linear relationships by adding X^2, X^3, ... as new features

How It Works

$$\hat{y} = w_0 + w_1X + w_2X^2 + w_3X^3 + \ldots$$

It's still LINEAR regression — linear in coefficients (w), not in X. We're just adding engineered features.

sklearn: PolynomialFeatures(degree=n) creates X, X^2, X^3, ... and all interaction terms automatically.

Performance by Polynomial Degree

Degree	# Features	Train R^2	Test R^2
1 (Linear)	13	0.751	0.669
2 (Poly)	104	0.941	0.806
3+	More	↑↑	↓↓

Overfitting: The Danger of High Degree

Underfitting (degree=1) → **Good Fit (degree=2-3)** → **Overfitting (degree=12+)**

Too simple
Misses patterns

Captures the real relationship

Memorizes noise
Fails on new data

Feature Scaling

Normalize feature ranges so regularization penalties are fair

Why Scale Features?

- Features on different scales (tax: 187–711 vs chas: 0–1)
- Plain Linear Regression: NOT affected by scaling
- Ridge/Lasso/Elastic Net PENALIZE large coefficients — unfair if features aren't on same scale
- **Rule: Fit on TRAIN, transform both train + test**

Scaler	Formula	Result	Best For
StandardScaler	$(X - \mu) / \sigma$	mean=0, std=1	Most cases, outliers
MinMaxScaler	$(X - \min) / (\max - \min)$	[0, 1]	Bounded values

Correct Scaling Workflow

scaler.fit(X_train) → **scaler.transform(X_train)**
scaler.transform(X_test)

Key Result from Notebook

Linear Regression (unscaled) Test $R^2 = 0.6688$

Linear Regression (scaled) Test $R^2 = 0.6688$

→ **Same! Scaling does NOT change plain LR results.**

→ **But it IS essential for Ridge, Lasso, and Elastic Net.**

Regularization: Ridge, Lasso & Elastic Net

Add a penalty term to the loss function to prevent overfitting

The Core Idea

```
OLS Loss:       Σ(yᵢ − ŷᵢ)²                                  ← No penalty, can overfit
Ridge Loss:     Σ(yᵢ − ŷᵢ)²  +  α Σwᵢ²                  ← L2: shrinks coefficients
Lasso Loss:     Σ(yᵢ − ŷᵢ)²  +  α Σ|wᵢ|                 ← L1: zeros out coefficients
Elastic Net:    Σ(yᵢ − ŷᵢ)²  +  α[ρΣ|wᵢ| + (1−ρ)Σwᵢ²]  ← L1+L2 combined
```

Property	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	$\alpha \Sigma w_i^2$	$\alpha \Sigma \lvert w_i \rvert$	$\alpha[\rho\Sigma\lvert w\rvert + (1-\rho)\Sigma w^2]$
Coefficients → 0?	Shrinks, never exactly 0	Can be exactly 0	Some zero, some shrunk
Feature selection?	No (keeps all)	Yes (drops some)	Yes (moderate)
Multicollinearity	Handles well	May drop 1 of corr pair	Keeps groups together
Best for	Many features, all useful	Sparse models, few important	Correlated groups

Ridge Regression (L2 Regularization)

Shrink all coefficients toward zero — but never eliminate them

How Ridge Works

$$\text{Loss} = \Sigma(y_i - \hat{y}_i)^2 + \alpha\ \Sigma w_i^2$$

- $\alpha = 0 \rightarrow$ plain Linear Regression
- α = large $\rightarrow$ heavy shrinkage, simpler model
- **All coefficients shrink but NONE become exactly 0**
- Handles multicollinearity well

Coefficient Path (in Notebook)

As α increases:

- All coefficients smoothly shrink toward 0
- Less important features shrink faster
- No coefficient ever hits exactly 0

$\rightarrow$ This is what makes Ridge different from Lasso!

Ridge Results

Ridge (α=1.0): Train R^2 = 0.7509 | Test R^2 = 0.6685

Best Ridge (α=5): Train R^2 = 0.7506 | Test R^2 = 0.6673 (via GridSearchCV)

Very similar to plain LR — Ridge shines more with high-dimensional or overfit models

Lasso Regression (L1 Regularization)

Shrink coefficients to EXACTLY zero — automatic feature selection

How Lasso Works

$$\text{Loss} = \Sigma(y_i - \hat{y}_i)^2 + \alpha\ \Sigma|w_i|$$

- L1 penalty uses absolute values |w|
- **Can force coefficients to EXACTLY 0**
- Effectively removes useless features
- **Built-in feature selection!**

Coefficient Path (in Notebook)

As α increases:

- Coefficients hit 0 one by one
- Least important features drop first
- Eventually only the most predictive remain

→ **This is Lasso's FEATURE SELECTION in action!**

Lasso Results

Lasso (α=0.1): Train R^2 = 0.7450 | Test R^2 = 0.6501

Best Lasso (α=0.01): Train R^2 = 0.7508 | Test R^2 = 0.6682 (via GridSearchCV)

With α=0.1, Lasso may drop features like indus and age — these are statistically insignificant (see p-values later)

Elastic Net (L1 + L2 Combined)

Best of both worlds: feature selection (Lasso) + stability (Ridge)

$$\text{Loss} = \Sigma(y_i - \hat{y}_i)^2 + \alpha[\rho\ \Sigma|w_i| + (1-\rho)/2\ \Sigma w_i^2]$$

α = overall regularization strength | ρ (l1_ratio) = L1 vs L2 mix: ρ=1 → Lasso | ρ=0 → Ridge | ρ=0.5 → equal

When to Use Elastic Net?

- Groups of correlated features
- Want feature selection + stability
- Lasso alone is too aggressive
- More features than samples (p > n)

Elastic Net Results

EN (α=0.1, l1=0.5): Test R^2 = 0.656

Best EN (α=0.01, l1=0.1): Test R^2 = 0.668

(via GridSearchCV over α and l1_ratio)

Ridge (L2)
l1_ratio = 0

→

Elastic Net
l1_ratio = 0.5

→

Lasso (L1)
l1_ratio = 1

Cross-Validation & Hyperparameter Tuning

Reliable model evaluation + automated search for the best α

5-Fold Cross-Validation

TEST	TRAIN	TRAIN	TRAIN	TRAIN
TRAIN	TEST	TRAIN	TRAIN	TRAIN
TRAIN	TRAIN	TEST	TRAIN	TRAIN
TRAIN	TRAIN	TRAIN	TEST	TRAIN
TRAIN	TRAIN	TRAIN	TRAIN	TEST

Each fold takes a turn as test set.

Final score = average of 5 runs ⟶ more reliable!

GridSearchCV — Automated Tuning

1. Define a grid of hyperparameter values

   `e.g., α = [0.001, 0.01, 0.1, 1, 10, 100]`

2. For each α, run 5-fold cross-validation
3. Pick the α with the highest average CV score

Results from Notebook:

- Best Ridge α = 5
- Best Lasso α = 0.01
- Best Elastic Net α = 0.01, l1_ratio = 0.1

Learning Curves (in Notebook)

- High bias: both curves low & converged ⟶ underfitting
- High variance: gap between train & test ⟶ overfitting
- Good model: both curves converge at a high R^2

Assumption Checking & Diagnostics

Linear Regression relies on 5 key assumptions — violating them makes results unreliable

#	Assumption	Meaning	How to Check	Violation Effect
1	Linearity	X and y have a linear relationship	Residual vs Fitted plot	Biased predictions
2	Independence	Observations are independent	Durbin-Watson test (≈2)	Invalid p-values
3	Normality	Residuals ~ Normal distribution	Q-Q plot, Shapiro-Wilk	CI & p-values wrong
4	Homoscedasticity	Constant residual variance	Residuals vs Fitted (flat band)	Inefficient estimates
5	No Multicollinearity	Features not highly correlated	VIF < 5 (ideal)	Unstable coefficients

4 Diagnostic Plots (in Notebook)

1. Residuals vs Fitted → Linearity + Homoscedasticity
2. Q-Q Plot → Normality of residuals
3. Histogram of residuals → Distribution shape
4. Scale-Location → Constant variance check

VIF — Multicollinearity Check

```
VIF = 1        → No correlation (good)
VIF = 1–5      → Moderate (usually OK)
VIF = 5–10     → High (investigate)
VIF > 10       → Severe (remove/regularize)
```

Ridge handles multicollinearity well!

Statistical Validation with Statsmodels

sklearn = prediction | statsmodels = inference (p-values, F-test, CI)

Reading the OLS Summary

Top Section:

- R^2 = proportion of variance explained
- Adj. R^2 = R^2 adjusted for # features
- F-statistic $p < 0.05$ → model is significant

Coefficient Table:

- coef = estimated coefficient value
- **P>|t| = p-value (< 0.05 = significant)**
- [0.025, 0.975] = 95% confidence interval

Bottom Section:

- Durbin-Watson ≈ 2.0 = no autocorrelation
- Omnibus/Jarque-Bera = normality tests
- Cond. No. very large → multicollinearity

Key Findings from Notebook

Hypothesis Testing for Each Feature:

H_0: coefficient = 0 (feature is useless)

H_1: coefficient ≠ 0 (feature matters)

If $p < 0.05$ → reject H_0 → feature is significant

Results:

- R^2 = 0.741, Adj. R^2 = 0.734
- F-stat p = 6.72e-135 (highly significant!)

- indus (p=0.738) → NOT significant
- age (p=0.958) → NOT significant
- All other features ARE significant

Lasso naturally dropped these insignificant features — matching the statistical analysis!

Evaluation Metrics Explained

How to measure and compare model performance

Metric	Formula	Interpretation	Range
R^2	$1 - SS_res / SS_tot$	% of variance explained by model	0 to 1 (1 = perfect)
Adj. R^2	R^2 penalized for # features	Fairer for comparing models with diff # features	0 to 1
RMSE	$\sqrt{(\Sigma(y-\hat{y})^2 / n)}$	Avg prediction error (same units as y)	0 to ∞ (0 = perfect)
MAE	$\Sigma\lvert y-\hat{y}\rvert / n$	Avg absolute error, less sensitive to outliers	0 to ∞ (0 = perfect)

Which Metric to Use?

- R^2: Good for overall model quality
- Adj. R^2: Better for model comparison
- RMSE: Penalizes large errors more
- MAE: More robust to outliers

- **Use Test metrics, not Train!**

Understanding R^2

$R^2 = 0.00 \rightarrow$ Model explains nothing

$R^2 = 0.50 \rightarrow$ Explains 50% of variance

$R^2 = 0.80 \rightarrow$ Good model!

$R^2 = 1.00 \rightarrow$ Perfect (suspicious—overfit?)

Train $R^2 \gg$ Test $R^2 \rightarrow$ Overfitting!

Final Model Comparison

Side-by-side performance of ALL techniques on the same test set

Model	Train R^2	Test R^2	RMSE	MAE
Polynomial LR (deg=2)	0.9409	0.8056	3.78	2.57
Multiple LR (all feat)	0.7509	0.6688	4.93	3.19
Ridge (α=1.0)	0.7509	0.6685	4.93	3.19
Best Lasso (α=0.01)	0.7508	0.6682	4.93	3.18
Best Elastic Net	0.7507	0.6676	4.94	3.18
Best Ridge (α=5)	0.7506	0.6673	4.94	3.18
Elastic Net (α=0.1)	0.7447	0.6563	5.02	3.19
Lasso (α=0.1)	0.7450	0.6501	5.07	3.24
Simple LR (lstat)	0.5423	0.5429	5.79	4.18

Winner: Polynomial LR (degree=2) — Test R^2 = 0.8056 | RMSE = 3.78

Captures non-linear relationships that plain LR misses. Regularization helps when dealing with many features / overfitting.

Key Takeaways & Recommended Workflow

Technique	Key Idea	When to Use
Simple LR	1 feature, 1 straight line	Baseline, single relationship
Multiple LR	All features, one equation	Multiple features available
Polynomial	Add X^2, X^3 for curves	Non-linear relationships
Ridge (L2)	Penalty on w^2	Multicollinearity, prevent overfit
Lasso (L1)	Penalty on $\|w\|$	Feature selection, sparse models
Elastic Net	L1 + L2 combined	Correlated feature groups

Recommended Workflow

1. Load & Explore Data
2. EDA (distributions, correlations)
3. Train-Test Split
4. Simple LR (baseline)
5. Multiple LR
6. Scale Features
7. Ridge / Lasso / Elastic Net
8. Cross-Validate & Tune
9. Check Assumptions
10. Compare & Select Best

Thank You!

Now open the notebook and run it hands-on!

Notebook: Linear Regression - Complete Guide with All Techniques.ipynb